Projet Wazuh – Détection d'intrusions

Informations

Étudiante : Amal LABIDI
Module : B2 Sécurité Pratique – TP1 / TP2
Détections : Déni de service HTTP (T1499), injection SQL (T1190), balayage de ports (T1046)
Plateforme : Wazuh 4.14 (manager + dashboard), machines Debian

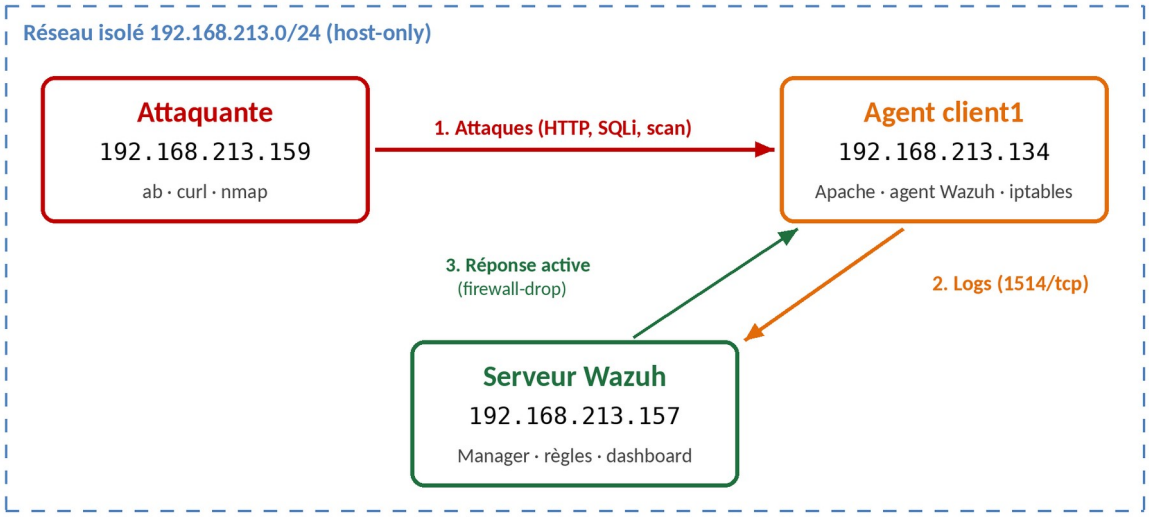
1. Contexte et objectif

À partir de la plateforme Wazuh des TP1 et TP2, je mets en place **trois détections d'intrusion**. Pour chacune : configuration, attaque depuis la machine de test, puis lecture de l'alerte dans le dashboard.

Détection	MITRE	Règle d'alerte	Réponse active
Déni de service HTTP	T1499	100201	Oui (blocage IP)
Injection SQL	T1190	31103 / 31106	Non
Balayage de ports Nmap	T1046	100301	Non

2. Schéma d'architecture

Machine	Rôle	Adresse IP
Serveur Wazuh	Manager + dashboard (règles, alertes)	192.168.213.157
Agent client1	Victime : Apache + agent Wazuh	192.168.213.134
Attaquante	Lance ab, curl, nmap	192.168.213.159

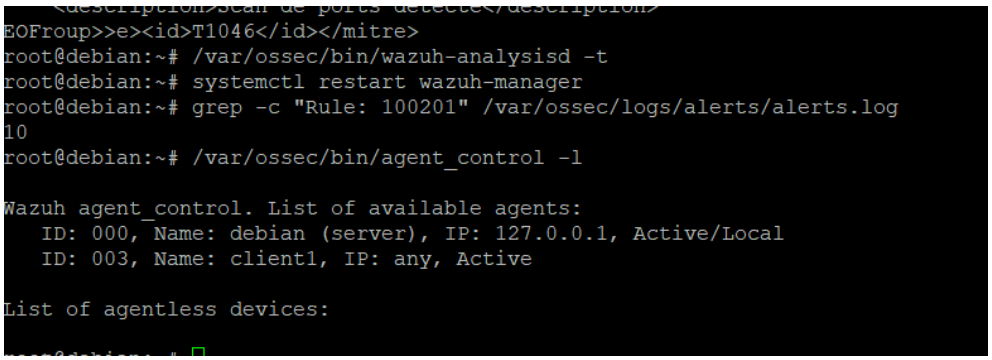


3. Préparation du lab

- Snapshot des 3 VM avant les tests.
- Réseau isolé (host-only), sans accès Internet.
- **Règle d'or : l'agent collecte les logs, le serveur porte les règles.**

Vérifier que l'agent est connecté (sur le serveur) :

/var/ossec/bin/agent_control -l



4. Détection 1 : Déni de service HTTP (MITRE T1499)

Explication

L'attaquant envoie un très grand nombre de requêtes pour saturer le serveur web. On alerte quand une même IP dépasse **100 requêtes en 10 secondes**, puis on la bloque.

Procédure

Étape 1 : Sur l'agent, lire le journal Apache (/var/ossec/etc/ossec.conf) :

```
<localfile>
    <log_format>apache</log_format>
    <location>/var/log/apache2/access.log</location>
</localfile>
```

Étape 2 : Sur le serveur, ajouter les règles (/var/ossec/etc/rules/local_rules.xml) :

```
<group name="local,web,dos,">
    <rule id="100200" level="3">
        <if_sid>31108</if_sid>
        <description>Requete web recue (base flood)</description>
    </rule>
    <rule id="100201" level="10" frequency="100" timeframe="10">
        <if_matched_sid>100200</if_matched_sid>
        <same_source_ip />
        <description>DoS HTTP : plus de 100 requetes en 10 s</description>
        <mitre><id>T1499</id></mitre>
    </rule>
</group>
```

Étape 3 : Sur le serveur, activer la réponse active (/var/ossec/etc/ossec.conf) :

```
<active-response>
    <command>firewall-drop</command>
    <location>local</location>
    <rules_id>100201</rules_id>
    <timeout>180</timeout>
</active-response>
```

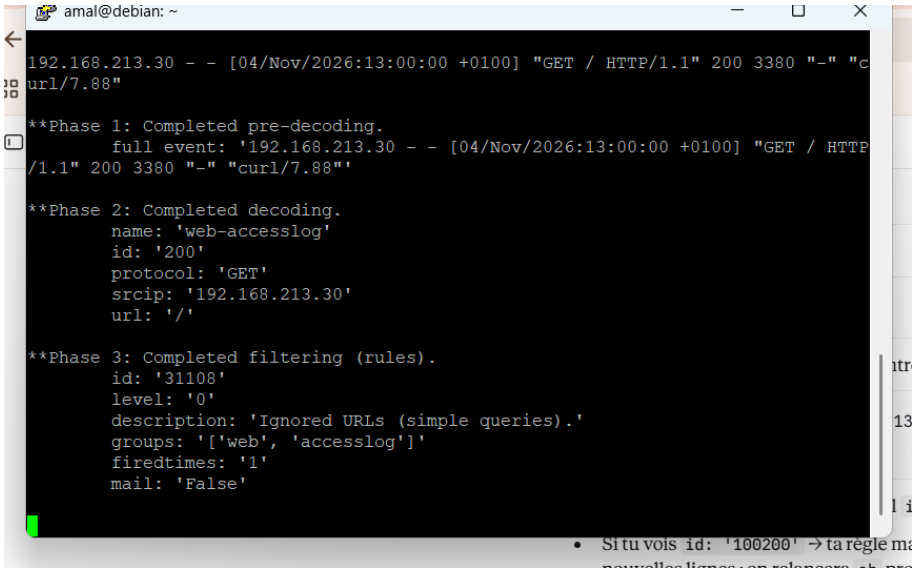
Étape 4 : Redémarrer le manager :

systemctl restart wazuh-manager

```
EOF
root@debian:~# systemctl restart wazuh-manager
grep -n "100200" /var/ossec/etc/rules/local_rules.xml
^C^C
32:  <rule id="100200" level="3">
37:    <if_matched_sid>100200</if_matched_sid>
root@debian:~# systemctl restart wazuh-manager
root@debian:~# grep -n "100200" /var/ossec/etc/rules/local_rules.xml
32:  <rule id="100200" level="3">
37:    <if_matched_sid>100200</if_matched_sid>
```

Avant

Une requête normale ne déclenche aucune alerte (règle 31108, niveau 0).



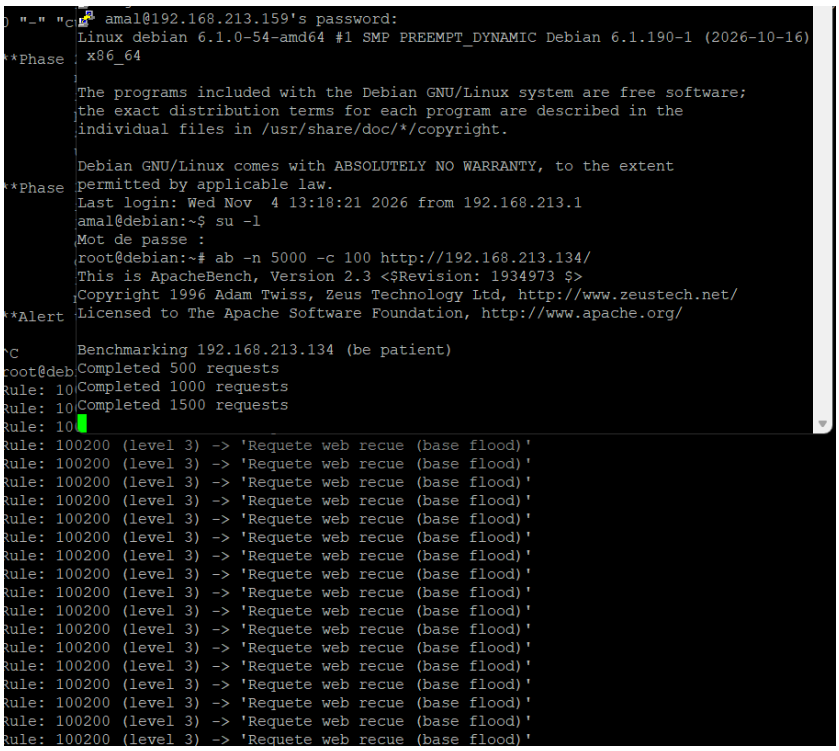
Pendant

Étape 5 : Depuis l'attaquante, lancer l'attaque :

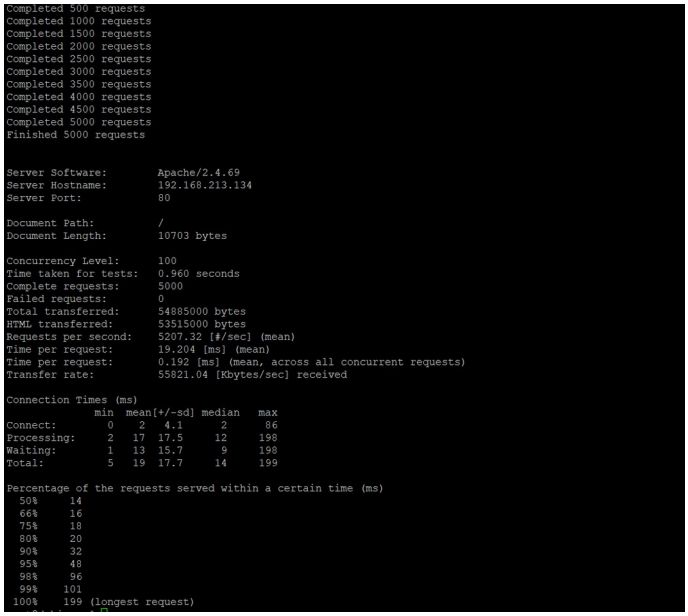
ab -n 5000 -c 100 http://192.168.213.134/

Étape 6 : Sur le serveur, suivre les alertes :

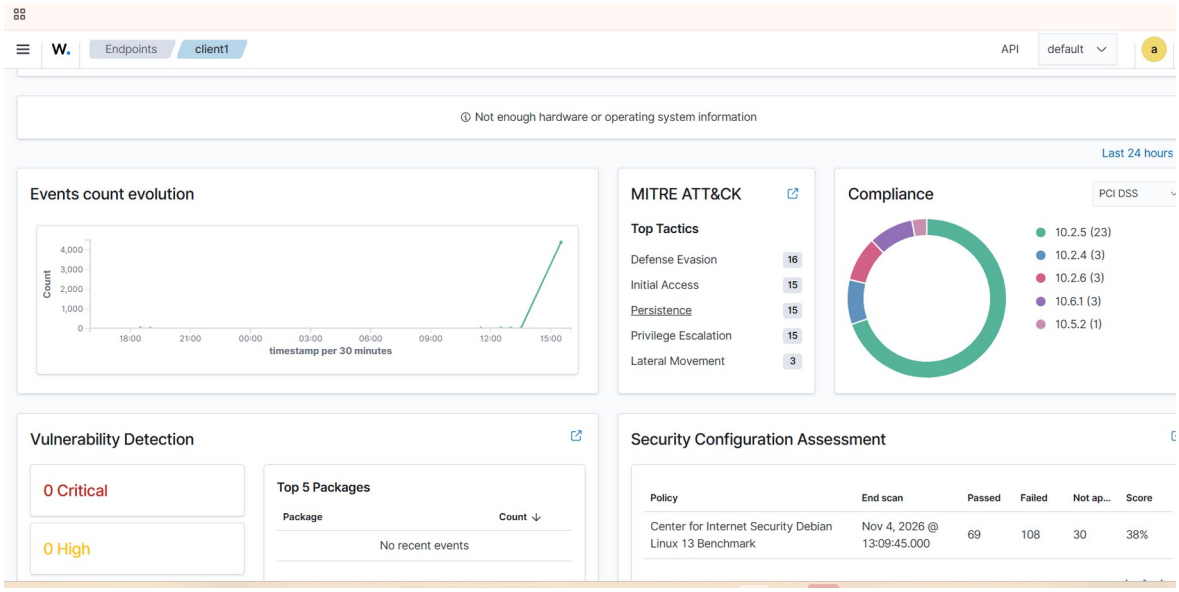
tail -f /var/ossec/logs/alerts/alerts.log | grep -E "100200|100201"

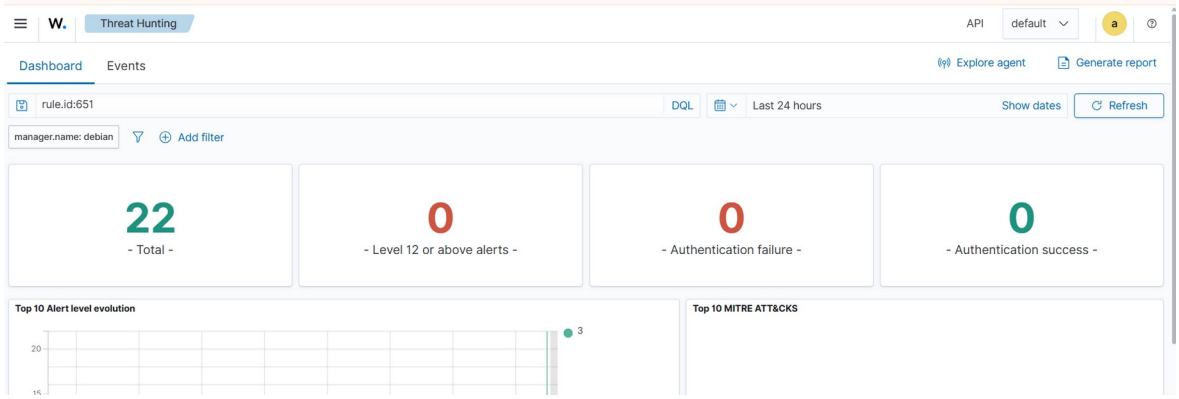


Résultat de ab : **5000 requêtes en 0,96 s**, soit environ **5200 requêtes par seconde**, bien au-dessus du seuil de 100 requêtes en 10 s.



Après





Vérifier le blocage sur l'agent :

iptables -L INPUT -n | grep 192.168.213.159

Juste après l'attaque, la machine attaquante ne peut plus joindre le serveur : un nmap lancé dans la foulée reste bloqué sans réponse, alors qu'il se termine normalement en moins d'une seconde. L'IP 192.168.213.159 est bien bloquée par la réponse active.

```
 100%    199 (longest request)
root@debian:~#  nmap -sT -p 1-1000 192.168.213.134
Starting Nmap 7.93 ( https://nmap.org ) at 2026-11-04 17:57 CET
```

Analyse

Règle **100201**, niveau **10**, MITRE **T1499**, IP source **192.168.213.159**. Après correction du doublon, **10 alertes 100201** ont été enregistrées. Elle déclenche firewall-drop : l'IP est bloquée 180 s (alerte **651**).

5. Détection 2 : Injection SQL (MITRE T1190)

Explication

L'attaquant place du code SQL dans un paramètre d'URL pour interroger la base de données. Wazuh le détecte **nativement** : aucune configuration à ajouter.

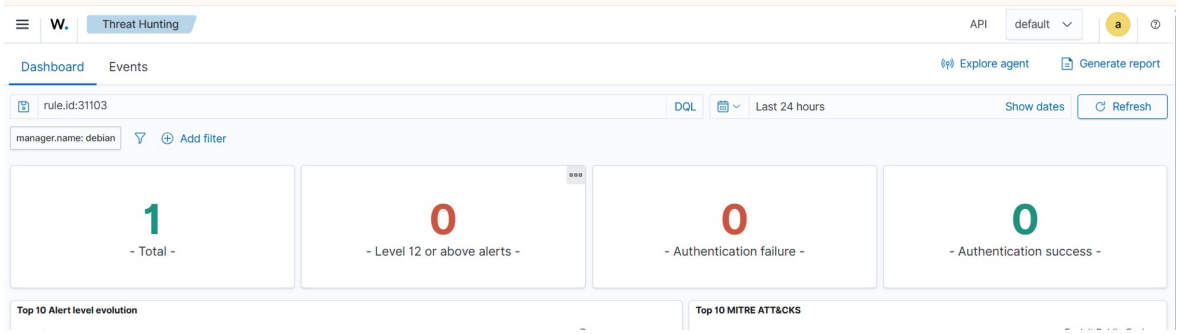
Procédure

Étape 1 : Depuis l'attaquante, envoyer la requête :

curl -XGET "http://192.168.213.134/users/?id=SELECT+*+FROM+users"

```
 100%   1110 (longest request)
root@debian:~# curl -XGET "http://192.168.213.134/users/?id=SELECT+*+FROM+users"
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/st
rict.dtd">
<html><head>
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL was not found on this server.</p>
<hr>
<address>Apache/2.4.69 (Debian) Server at 192.168.213.134 Port 80</address>
</body></html>
root@debian:~#
```

Étape 2 : Dans le dashboard (Threat Hunting), filtrer : rule.id:31103



Analyse

Règle **31103** « SQL injection attempt », niveau **7**, MITRE **T1190**. La règle **31106** (niveau 12) ne s'est pas déclenchée car le serveur a répondu **404** : l'attaque est détectée mais n'a pas réussi.

6. Détection 3 : Balayage de ports Nmap (MITRE T1046)

Explication

L'attaquant teste de nombreux ports pour trouver les services ouverts. Le pare-feu de l'agent journalise chaque connexion ; on alerte au-delà de **15 en 5 secondes** depuis une même IP.

Procédure

Étape 1 : Sur l'agent, installer les outils et journaliser les connexions :

apt install iptables rsyslog
iptables -I INPUT -p tcp --syn -j LOG --log-prefix "SCAN-DETECT: "

Étape 2 : Sur l'agent, lire kern.log (/var/ossec/etc/ossec.conf), puis redémarrer l'agent :

```
<localfile>
    <log_format>syslog</log_format>
    <location>/var/log/kern.log</location>
</localfile>
```

Étape 3 : Sur le serveur, ajouter les règles (local_rules.xml) puis redémarrer le manager :

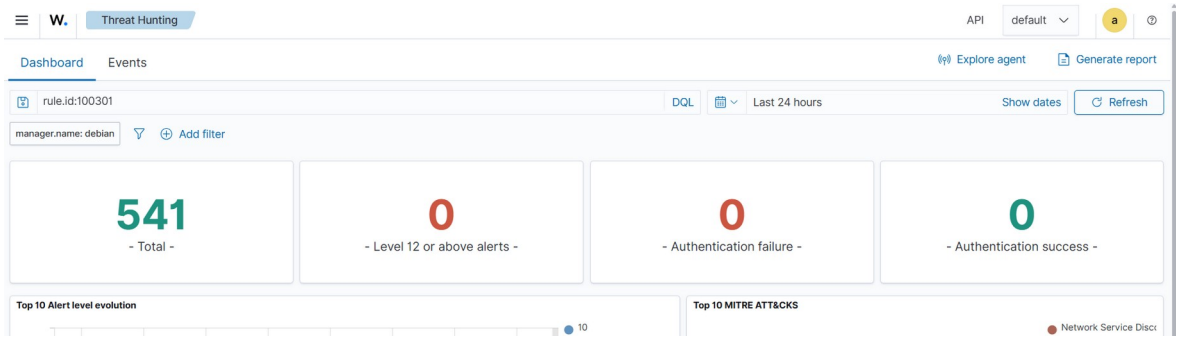
```
<group name="local,scan,">
    <rule id="100300" level="3">
        <if_sid>4100</if_sid>
        <match>SCAN-DETECT</match>
        <description>Connexion journalisee par iptables</description>
    </rule>
    <rule id="100301" level="10" frequency="15" timeframe="5">
        <if_matched_sid>100300</if_matched_sid>
        <same_source_ip />
        <description>Scan de ports detecte</description>
        <mitre><id>T1046</id></mitre>
    </rule>
</group>
```

Pendant

Étape 4 : Depuis l'attaquante, lancer le scan :

nmap -sT -p 1-1000 192.168.213.134

Après



Analyse

Règle **100301**, niveau **10**, MITRE **T1046**, IP source **192.168.213.159**. 1000 ports testés en 0,24 s : le seuil est dépassé immédiatement.

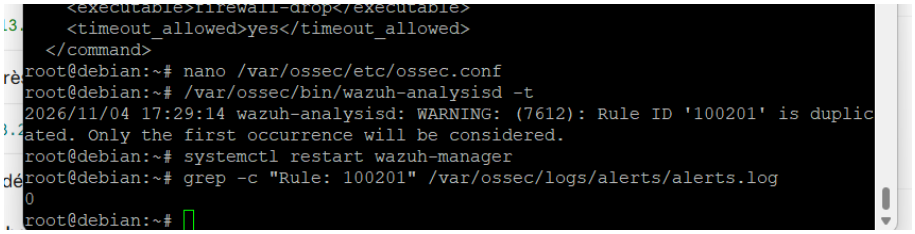
7. Analyse des alertes

Règle	Niveau	Signification	MITRE
100201	10	DoS HTTP : > 100 requêtes / 10 s	T1499
31103 / 31106	7 / 12	Tentative d'injection SQL / attaque réussie (code 200)	T1190
100301	10	Scan de ports : > 15 connexions / 5 s	T1046
651	3	IP bloquée par la réponse active	—

8. Difficultés rencontrées et solutions

Difficulté	Solution
Pas d'Internet (réseau isolé)	NAT temporaire pour installer les paquets
Confusion agent / serveur	wazuh-control info | grep type

Difficulté	Solution
Règle DoS ignorée (web-accesslog est un décodeur, pas un groupe)	Utiliser if_sid 31108
L'agent ne remontait pas les logs	Redémarrer wazuh-agent
Manager planté (décodeur iptables inexistant + doublons)	Tester avec wazuh-analysisd -t avant de redémarrer
Pas d'alerte 100201 : règle écrite deux fois (« Rule ID is duplicated »)	Supprimer le doublon dans local_rules.xml, revérifier avec wazuh-analysisd -t
kern.log absent (journald, pas rsyslog)	Installer rsyslog
Règle scan sans résultat (la règle 4100 gagnait)	if_sid 4100 + niveau 3



9. Conclusion

Les trois détections fonctionnent : le DoS est détecté et bloqué automatiquement, l'injection SQL est reconnue par les règles natives, et le scan Nmap devient visible grâce aux logs iptables. J'ai surtout retenu la séparation agent (collecte) / serveur (règles) et l'importance de tester la configuration avant de redémarrer.